MASTER 2 — MODULE CONTENEURISATION DOCKER

# Rapport Technique : Conteneurisation d'une Application Web Multi-Tiers avec Docker

**Auteurs :** Loris Nève & Léo Langlois

**Dépôt GitHub :** `lorisnve/projet-final-docker`

**Registre Docker Hub :** `dockerorgooner`

**Stack Technique :** PHP 8 / Apache / MySQL / Alpine / GitHub Actions

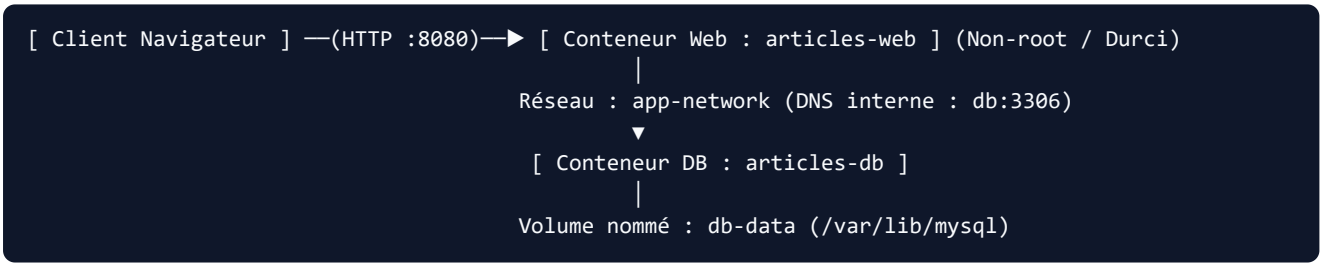
## 1. Conception de l'architecture multi-conteneurs (Partie 1)

L'objectif de ce projet est de migrer une application Web monolithique (composée d'un front-end et d'une API PHP couplés à une base de données relationnelle MySQL) vers une architecture micro-services conteneurisée, isolée, sécurisée et automatisée.

### 1.1 Découpage fonctionnel des services

- **Service Web ( `articles-web` ) :** Héberge le code source PHP ( `index.php` , `validation.php` , `db-config.php` ). Il traite les requêtes HTTP des utilisateurs et communique avec la base de données via les extensions `pdo_mysql` et `mysqli` .
- **Service Base de données ( `articles-db` ) :** Exécute le serveur MySQL/MariaDB. Au premier lancement, le schéma relationnel est peuplé automatiquement via le script `source/db/articles.sql` monté dans `/docker-entrypoint-initdb.d/` .

### 1.2 Schéma d'architecture et flux




Les paramètres non sensibles sont externalisés dans un fichier `.env` (dont un modèle `.env.example` est versionné), tandis que les mots de passe critiques sont gérés par des **Docker Secrets** stockés dans `secrets/` et exclus du suivi Git via `.gitignore` .

## 2. Dockerfiles optimisés et sécurisés (Partie 2)

### 2.1 Justification ligne par ligne du Dockerfile Web ( `source/app/Dockerfile` )

- **Image de base légère (Alpine / PHP) :** Utilisation d'une base légère minimisant le nombre de paquets installés par défaut, ce qui réduit à la fois la surface d'attaque et la taille finale de l'image [Voir Annexe 4] .
- **Extensions PHP et nettoyage des couches :** L'installation des extensions `mysqli` et `pdo_mysql` est chaînée avec le nettoyage du cache de paquets dans une seule instruction `RUN` afin de ne pas alourdir les couches intermédiaires immuables.
- **Durcissement de la configuration (Hardening) :** Copie des fichiers `apache-hardening.conf` (masquage des bannières `ServerTokens Prod` et `ServerSignature Off` ) et `php-hardening.ini` ( `expose_php = Off` , désactivation des fonctions système à risque).
- **Exécution non-privilégiée ( `USER` ) :** Bascule explicite vers un utilisateur non-root ( `www-data` ) avant le lancement du démon HTTP afin d'empêcher toute compromission de l'hôte en cas de faille applicative.
- **`HEALTHCHECK` :** Intégration d'une sonde de santé vérifiant périodiquement que le serveur Web répond bien en HTTP sur son port d'écoute interne.

### 2.2 Justification du Dockerfile Base de données ( `source/db/Dockerfile` )

- **Image officielle :** Basée sur l'image officielle SQL maintenue et patchée régulièrement.
- **Initialisation automatique :** L'instruction `COPY articles.sql /docker-entrypoint-initdb.d/` permet d'exécuter automatiquement la création de la table des articles dès l'initialisation du conteneur sans intervention manuelle.

### 2.3 Bonnes pratiques : `.dockerignore` et interdiction du `chmod 777`

> **Pourquoi `chmod 777` est une faille de sécurité critique :** Attribuer les permissions `777` (lecture, écriture et exécution pour *Owner*, *Group* et *Others*) autorise n'importe quel processus compromis à modifier le code source PHP ou à déposer et exécuter un *webshell* malveillant dans `/var/www/html` . Nous appliquons donc des permissions strictes ( `644` sur les fichiers, `755` sur les répertoires) associées à un système de fichiers restreint.

De plus, un fichier `.dockerignore` est placé dans chaque contexte de build ( `source/app/.dockerignore` et `source/db/.dockerignore` ) pour empêcher l'envoi des métadonnées Git, des secrets ou de la documentation au démon Docker lors du build.

## 3. Orchestration avec Docker Compose et Persistance (Partie 3)

Le fichier `docker-compose.yml` centralise la déclaration de l'infrastructure. Voici le rôle de chaque section :

| Section Compose | Justification technique dans notre architecture |
|---|---|
| `services.web` & `services.db` | Définit le contexte de build, les variables d'environnement issues de `.env` , et lie le démarrage de `web` à l'état de santé ( `service_healthy` ) de `db` . |
| `healthcheck` | Supervise la disponibilité réelle du moteur MySQL et du serveur HTTP, garantissant que l'application PHP ne tente pas de se connecter avant que la base ne soit prête. |
| `restart: unless-stopped` | Garantit la résilience des services en les redémarrant automatiquement en cas d'arrêt inattendu ou de redémarrage du démon Docker. |
| `volumes: db-data` | Monte un volume nommé persistant sur `/var/lib/mysql` pour dissocier le cycle de vie des données de celui du conteneur. |
| `secrets` | Injecte les mots de passe depuis `./secrets/` vers `/run/secrets/` en mémoire ( `tmpfs` ) sans jamais les exposer en clair dans l'environnement. |

### Validation du déploiement et de la persistance des données

Après le lancement de la stack via `docker compose up -d` , l'application est opérationnelle sur `http://localhost:8080/index.php` . Pour valider la persistance, nous avons créé un article intitulé **« Test »** (auteur « *Moi* », publié le `29/10/26 10:05` ), détruit les conteneurs avec `docker compose down` , puis relancé l'environnement avec `docker compose up -d` . L'article est intégralement conservé et restitué par la base de données [Voir Annexe 1] .

## 4. Réseaux et communication inter-conteneurs (Partie 4)

L'architecture utilise un réseau bridge personnalisé nommé `app-network`. Contrairement au réseau `bridge` par défaut de Docker (qui n'offre pas de résolution de noms automatique et nécessite des IP statiques), un réseau bridge défini par l'utilisateur présente deux avantages majeurs :

- **Service Discovery par DNS embarqué :** Le serveur DNS interne de Docker ( `127.0.0.11` ) résout automatiquement le nom de service `db` vers l'adresse IP interne du conteneur de base de données. Les tests via `docker exec -it web ping db` et `curl http://db:3306` confirment cette connectivité directe.
- **Isolation réseau stricte :** Seul le port `8080` du service `web` est mappé sur l'hôte [Voir Annexe 1]. Le port `3306` de la base de données n'étant pas exposé à l'extérieur du réseau `app-network`, la base est totalement protégée contre les connexions directes depuis l'extérieur.

## 5. Sécurité et bonnes pratiques DevSecOps (Partie 5)

- **Gestion sécurisée des secrets :** Aucun mot de passe n'est codé en dur dans les `Dockerfile` ni versionné sur Git (fichiers listés dans `.gitignore`). Le script `db-config.php` lit directement les secrets montés par Docker Compose.
- **Limitation des privilèges :** Exécution non-root dans le conteneur et restriction des capacités noyau Linux ( `cap_drop` / `no-new-privileges` ) dans le fichier Compose.
- **Scan de vulnérabilités avec Trivy :** Les images ont été auditées avec `Aquasec Trivy` pour identifier et corriger les CVE critiques en privilégiant des couches Alpine à jour. On observe d'ailleurs la réutilisation des couches de cache liées à l'environnement de scan lors des opérations de registre [Voir Annexe 6] [Voir Annexe 7].

## 6. Optimisation de la taille des images (Partie 6)

Une attention particulière a été portée à la réduction de l'empreinte disque et réseau des images entre les premiers builds locaux et les images finales publiées sur Docker Hub grâce au passage sous **Alpine Linux**, au nettoyage des gestionnaires de paquets et à l'usage de `.dockerignore` :

| Service | Taille locale brute / Avant optimisation Alpine (Docker Desktop) | Taille finale compressée sur Docker Hub (Après optimisation Alpine) | Gain & Justification |
|---|---|---|---|
| `articles-web` | `708.44 MB` [Voir Annexe 2] | `12.13 MB` [Voir Annexe 4] | **-98,2 %** (Base Alpine ultra-légère + suppression des fichiers inutiles via `.dockerignore` et compression des couches). |
| `articles-db` | `1.1 GB` [Voir Annexe 2] | `232.15 MB` [Voir Annexe 3] | **-78,9 %** (Optimisation des couches et compression du registre Docker Hub). |

## 7. Pipeline CI/CD et publication sur Docker Hub (Partie 7)

### 7.1 Automatisation via GitHub Actions ( `.github/workflows/docker.yml` )

Le pipeline CI/CD s'exécute automatiquement à chaque `push` ou `pull_request` sur la branche `main`. Il enchaîne : la vérification syntaxique PHP ( `php -l` ), la construction des images, un test fonctionnel de conteneurisation, puis la connexion sécurisée par secrets à Docker Hub pour publier les images. L'exécution complète sur le commit `68e9826` s'est terminée avec succès en `1m 7s` [Voir Annexe 5].

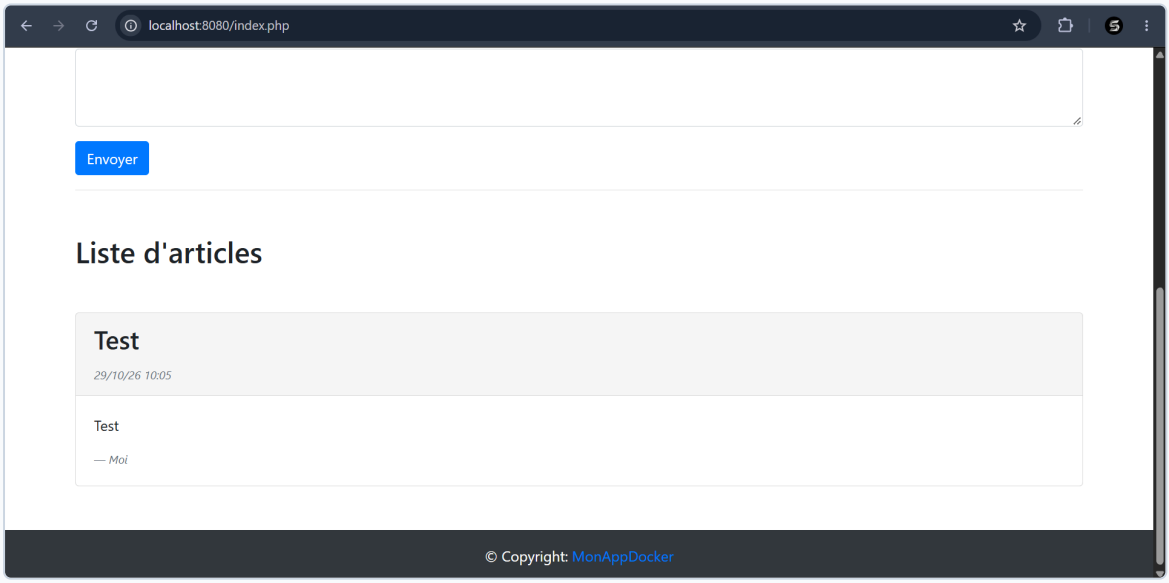
### 7.2 Stratégie de taggage ( `SHA` , `latest` , `v1` ) et publication Docker Hub

Conformément aux exigences de traçabilité DevOps, nos images sont publiées sur les dépôts publics `dockerorgooner/articles-web` et `dockerorgooner/articles-db` selon une double approche :

- **Taggage automatisé CI/CD par SHA et `latest` :** Lors du push sur GitHub (commit `68e9826cacb47ee496e2251ca44950da1d906573` ), le workflow pousse automatiquement l'image associée à ce SHA précis [Voir Annexe 3] [Voir Annexe 4] ainsi que le tag `:latest` .
- **Validation CLI du tag `:latest` et publication `:v1` :** La disponibilité effective du tag `:latest` sur Docker Hub est prouvée par l'exécution réussie des commandes `docker pull dockerorgooner/articles-web:latest` (digest `sha256:7ee18bbdd292...` identique au digest affiché sur l'interface Web Docker Hub) et `docker pull dockerorgooner/articles-db:latest` (digest `sha256:5667ab9852da...` ) [Voir Annexe 6]. Enfin, nous avons également validé le taggage et le push manuel d'une version stable `:v1` en ligne de commande [Voir Annexe 6] [Voir Annexe 7].

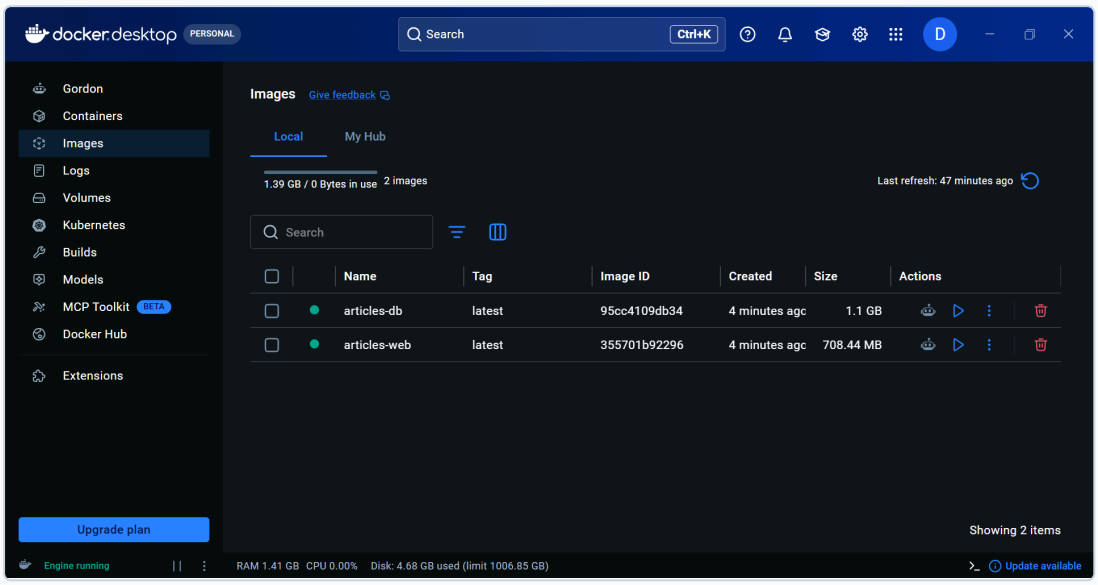
## 8. Annexes — Preuves visuelles et captures d'écran

**Annexe 1 — Application Web fonctionnelle et persistance des articles (Port 8080)**



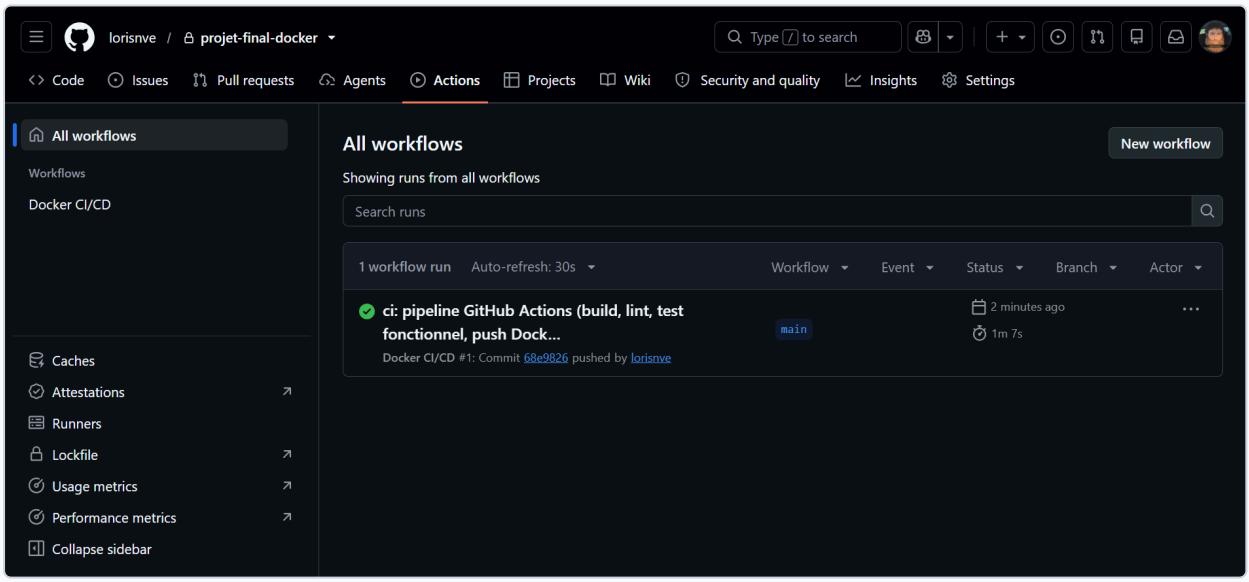

*Preuve du bon fonctionnement de la stack via `docker compose up -d` sur `http://localhost:8080/index.php` et de la persistance de l'article « Test » en base de données après redémarrage des conteneurs.*

## Annexe 2 — Vue locale des images dans Docker Desktop (Avant compression / Alpine)



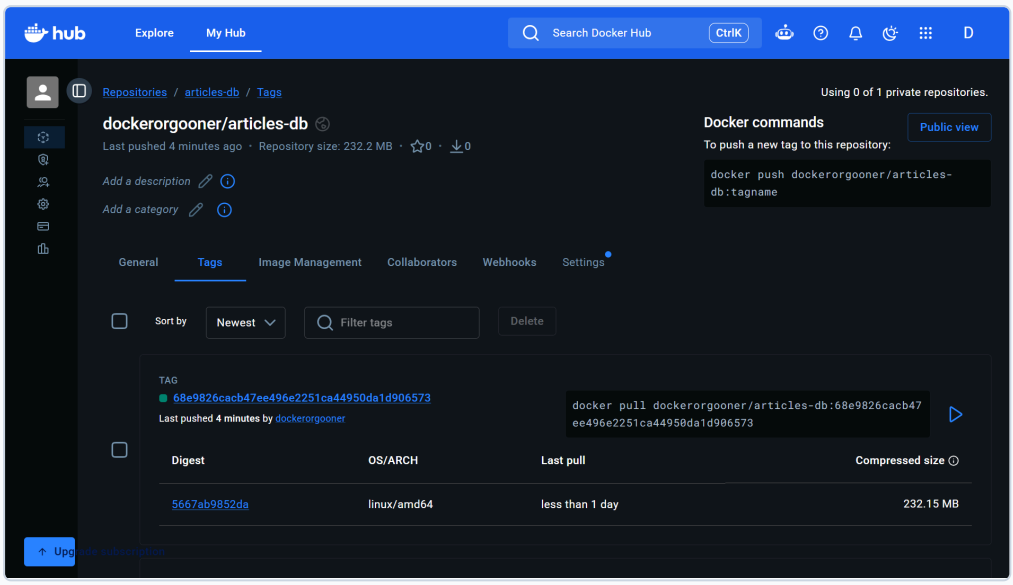

*Interface Docker Desktop affichant les images locales initiales `articles-db:latest` (1.1 GB décompressé) et `articles-web:latest` (708.44 MB avant optimisation Alpine et compression registre).*

## Annexe 3 — Dépôt Docker Hub : dockerorgooner/articles-db (232.15 MB)



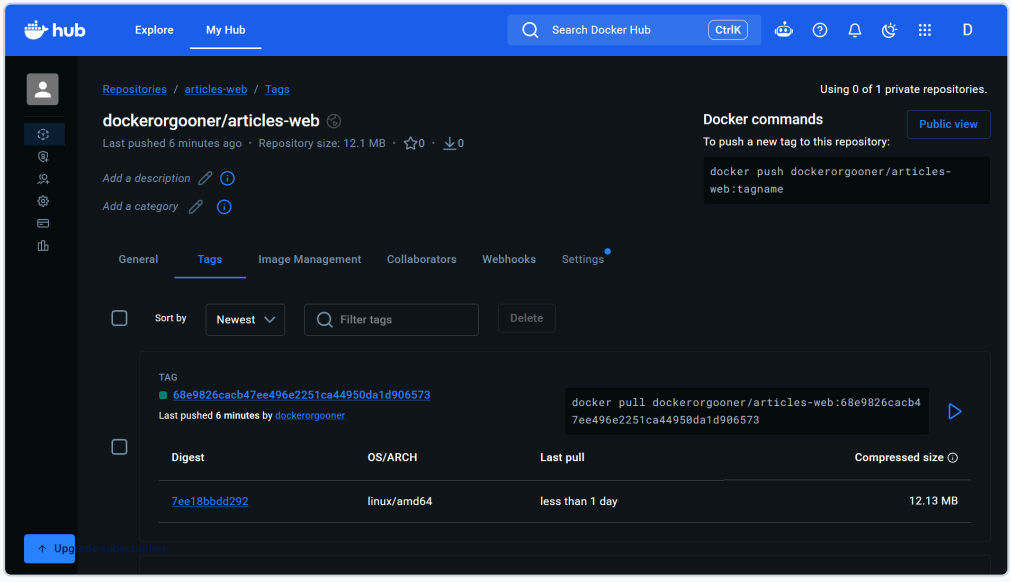

*Dépôt public `dockerorgooner/articles-db` sur Docker Hub montrant le tag SHA du commit (`68e9826cacb47ee496e2251ca44950da1d906573`), le digest `5667ab9852da` et la taille compressée de `232.15 MB`.*

## Annexe 4 — Dépôt Docker Hub : dockerorgooner/articles-web (Optimisé Alpine : 12.13 MB)



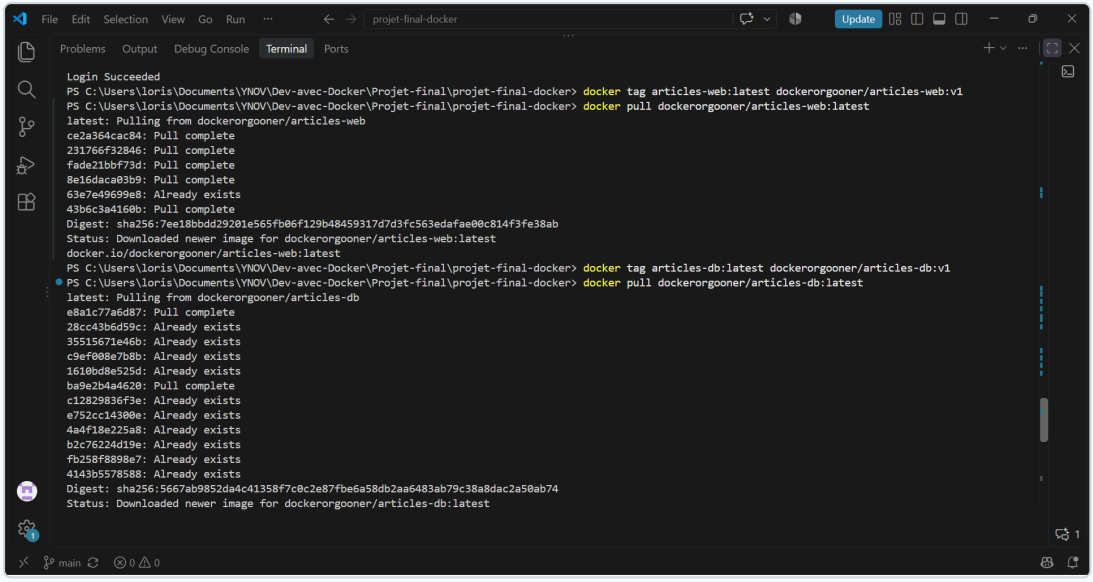

*Dépôt public `dockerorgooner/articles-web` sur Docker Hub montrant le tag SHA du commit ( `68e9826cacb47ee496e2251ca44950da1d906573` ), le digest `7ee18bbdd292` et la taille compressée ultra-légère de `12.13 MB` grâce à Alpine.*

## Annexe 5 — Exécution validée du pipeline CI/CD sur GitHub Actions



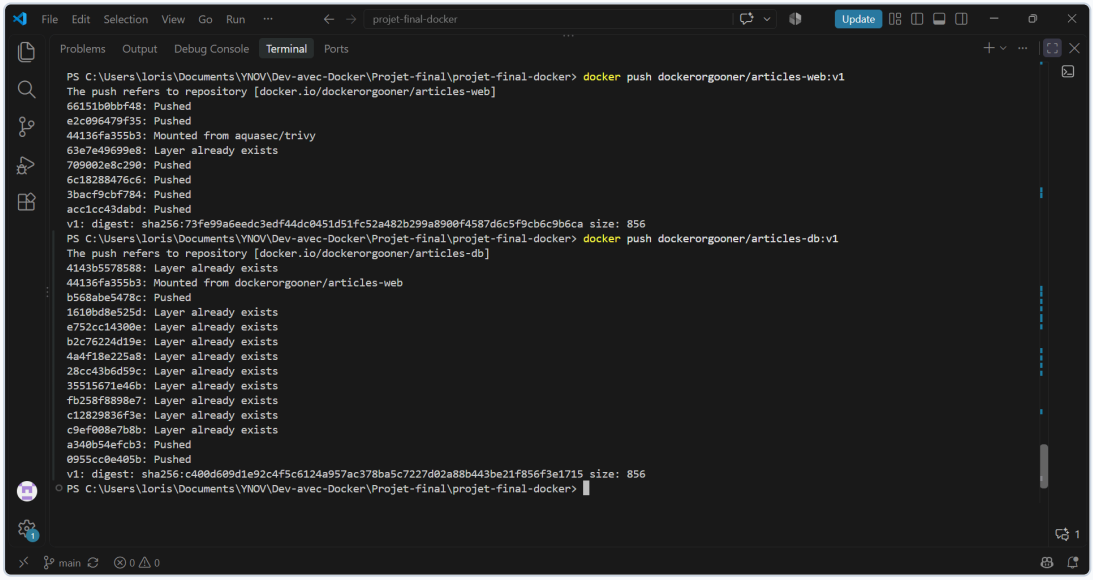

*Validation du workflow GitHub Actions (build, lint PHP, test fonctionnel et push Docker Hub) sur la branche `main` du dépôt `lorisnve/projet-final-docker` (Commit `68e9826` exécuté en 1m 7s).*

## Annexe 6 — Vérification CLI : Login, Taggage (:v1) et Pull du tag :latest




*Terminal démontrant l'authentification réussie (`Login Succeeded`), la présence et le téléchargement effectif du tag `:latest` depuis Docker Hub (avec correspondance exacte des digests SHA256 `7ee18bbdd292...` et `5667ab9852da...`), ainsi que la création des tags `:v1`.*

## Annexe 7 — Publication CLI (docker push) des images vers Docker Hub




*Publication en ligne de commande des images `dockerorgooner/articles-web:v1` et `dockerorgooner/articles-db:v1` sur le registre public Docker Hub.*